

UNIVERSITY OF SALERNO
DEPARTMENT OF COMPUTER SCIENCE

Hardware-Based Authentication via SRAM PUF on ESP32 Microcontrollers

IoT Security

Authors:

Amedeo LEONE 0522501950
Rocco PAGLIARULO 0522502086

June 27, 2026



Contents

1	Project Overview and Objectives	2
1.1	Context and Problems	2
1.2	Proposed Solution	2
1.3	Key Project Objectives	2
1.4	Functional Requirements (FR)	2
1.5	Non-Functional Requirements (NFR)	3
2	System Architecture	4
2.1	Edge Layer: Microcontroller (ESP32)	4
2.2	Gateway and Validation Layer: Node-RED	4
2.3	Offline Analysis Suite (Python Scripts)	6
3	Data Flow and Security	6
3.1	Hardware Purification	6
3.2	Payload Encapsulation	7
3.3	Transmission and Decryption	7
3.4	Fault-Tolerant Validation (Hamming Distance)	7
4	Setup and Deployment Guide	8
4.1	Hardware and Software Prerequisites	8
4.2	Deployment Procedure	8
5	Known Issues and Future Developments	8
5.1	Known Issues (Current State)	9
5.2	Next Steps (Backlog for Upcoming Sprints)	9

1 Project Overview and Objectives

1.1 Context and Problems

In modern Internet of Things (IoT) scenarios, the secure authentication of edge nodes represents a critical architectural challenge. Microcontrollers frequently operate in distributed and unsupervised environments, making them vulnerable to physical attacks. Traditionally, the identity of a node is guaranteed by storing pre-shared cryptographic keys in non-volatile memories (Flash or EEPROM). This approach exposes the system to serious risks: an attacker with physical access to the device can extract the keys, clone the node, and compromise the entire network.

1.2 Proposed Solution

To mitigate these vulnerabilities, this project implements a hardware-based authentication mechanism leveraging Physical Unclonable Functions (PUFs). Instead of storing a static key, the system exploits the intrinsic unpredictability and microscopic manufacturing variations of the silicon within the ESP32 microcontroller. Specifically, the project utilizes a Cold Boot-based SRAM PUF: upon power-up, the initial state of the memory cells (which tend randomly toward 0 or 1) provides a unique "fingerprint" for every single chip, which is impossible to physically clone or mathematically predict.

1.3 Key Project Objectives

- **Zero-Key Storage:** Generate a volatile authentication key "on the fly" during device startup by reading an isolated block of memory (RTC RAM), thus avoiding any permanent storage on the edge node.
- **On-Edge Processing and Purification:** Implement lightweight algorithms directly on the microcontroller (such as Von Neumann debiasing) to clean the raw memory dump, removing redundancies before transmission.
- **Secure Transmission (Confidentiality):** Protect the payload in transit over the Wi-Fi network by encapsulating the purified PUF key within a JSON packet, entirely encrypted using the AES-128 symmetric cryptographic standard in ECB mode.
- **Fault-Tolerant Validation:** Develop a validation gateway (via Node-RED) capable of authenticating the device by calculating the Hamming Distance between the received PUF and a reference "Master PUF." This approach allows for the tolerance of natural electronic noise in SRAM readings (guaranteeing access under a specific error threshold) and deterministically rejecting any clone devices.

1.4 Functional Requirements (FR)

The functional requirements describe the specific operations that the system and its subsystems must be able to perform.

FR1 - Hardware Fingerprint Extraction: The edge node must be able to extract 1024 bytes of raw data from an SRAM memory area (SRAM PUF) upon device power-up (Cold Boot).

- FR2 - Data Purification (Debiasing):** The microcontroller must locally apply the Von Neumann algorithm to balance the extracted binary string, discarding redundant bit pairs (00 and 11) and converting valid pairs (01 to 0, 10 to 1).
- FR3 - Payload Encryption:** The edge node must format the purified fingerprint into a JSON packet and encrypt it using the AES-128 cryptographic standard in ECB mode. The system must apply PKCS#7 padding to adjust the text length to the 16-byte blocks required by the algorithm.
- FR4 - Data Transmission:** The device must establish a local Wi-Fi connection and send the encrypted payload to the central server via an HTTP POST request (endpoint /auth).
- FR5 - Server Validation and Authentication:** The Node-RED server must receive the payload, decrypt it using a pre-shared 16-character key (PSK), and compare the received PUF with a reference "Master" fingerprint.
- FR6 - Acceptance Logic:** The central system must calculate the Hamming Distance between the two keys. Access is guaranteed exclusively if the error percentage (discrepancy) is strictly lower than the established 15.0% threshold. Otherwise, access must be denied and the payload labeled as a cloning attempt.
- FR7 - Offline Analysis Tools:** The project must include a software suite for offline analysis (Python scripts) capable of quantifying key metrics such as reliability (Intra-Distance, tolerance $\geq 40\%$) and uniqueness (Inter-Distance, between 40% and 60%) of memory dumps extracted from different chips.

1.5 Non-Functional Requirements (NFR)

The non-functional requirements define the quality, security, and performance constraints of the system.

- NFR1 - Data Transit Security:** No sensitive data (e.g., cleartext PUF fingerprint or alarm messages) must transit the network without protection. Interception of the Wi-Fi packet must not compromise the system.
- NFR2 - Memory Integrity:** The RTC memory area used for PUF extraction must be explicitly isolated (via the RTC_NOINIT_ATTR attribute) to prevent the device bootloader from overwriting the initial electrical state of the registers before reading.
- NFR3 - Computational Efficiency:** The debiasing (Von Neumann) and block encryption processes must be lightweight enough to be executed directly on the microcontroller during startup without causing blocking delays.
- NFR4 - Server Usability:** The Dashboard interface on Node-RED must provide immediate visual feedback on the authentication status, displaying the cleartext message (if authorized) and the exact calculated discrepancy percentage (Hamming Distance) in real-time.

2 System Architecture

The system is based on a three-tier architecture: the edge node (responsible for hardware extraction and client-side security), the server/gateway (responsible for validation), and an offline suite for statistical and cryptographic analysis.

2.1 Edge Layer: Microcontroller (ESP32)

The core of hardware identity extraction lies in the edge node, programmed in C++ using the Arduino environment (`PUF_starter_1.ino`). The ESP32 performs the following critical operations during Cold Boot:

- **PUF Extraction:** The device reads 1024 bytes of raw data from an isolated memory area (`RTC_NOINIT_ATTR`), preserving it from being overwritten by the bootloader during startup.
- **On-Chip Conversion and Debiasing:** Raw hexadecimal data is converted into binary and processed internally via the Von Neumann algorithm to remove systematic biases (discarding the 00 and 11 bit pairs).
- **Payload Encryption:** The purified PUF key is inserted into a plaintext JSON payload (along with a test message, e.g., "Temperature Alarm!"). This payload is encrypted using the `mbedtls` library via the AES-128 algorithm in ECB mode with PKCS#7 padding, using a 16-character pre-shared key (PSK) ("SuperSeg-reta1234").
- **Network Transmission:** The microcontroller connects to a Wi-Fi network (SSID: "Pixel") and sends only a new JSON containing the encrypted hexadecimal string to the server's HTTP endpoint. This operation is performed exactly once at startup within the `setup()` function.

2.2 Gateway and Validation Layer: Node-RED

The backend and user interface are implemented in Node-RED (`flows-2.json`), serving as both an API endpoint and an operational dashboard.

- **Reception Endpoint:** An `http in` node listens on the `/auth` path to receive incoming POST requests from the ESP32.
- **Decryption (`crypto-js` module):** Within a dedicated function node, the system extracts the encrypted payload and uses the `crypto-js` library with the same secret key (AES-128 ECB, PKCS#7) to decrypt the data and reconstruct the original JSON.
- **Authentication Engine (Hamming Distance):** The server extracts the decrypted PUF sent by the edge node and compares it bit-by-bit with a "Master PUF" hardcoded into the system. The engine calculates the Hamming Distance between the two strings and determines the error percentage.

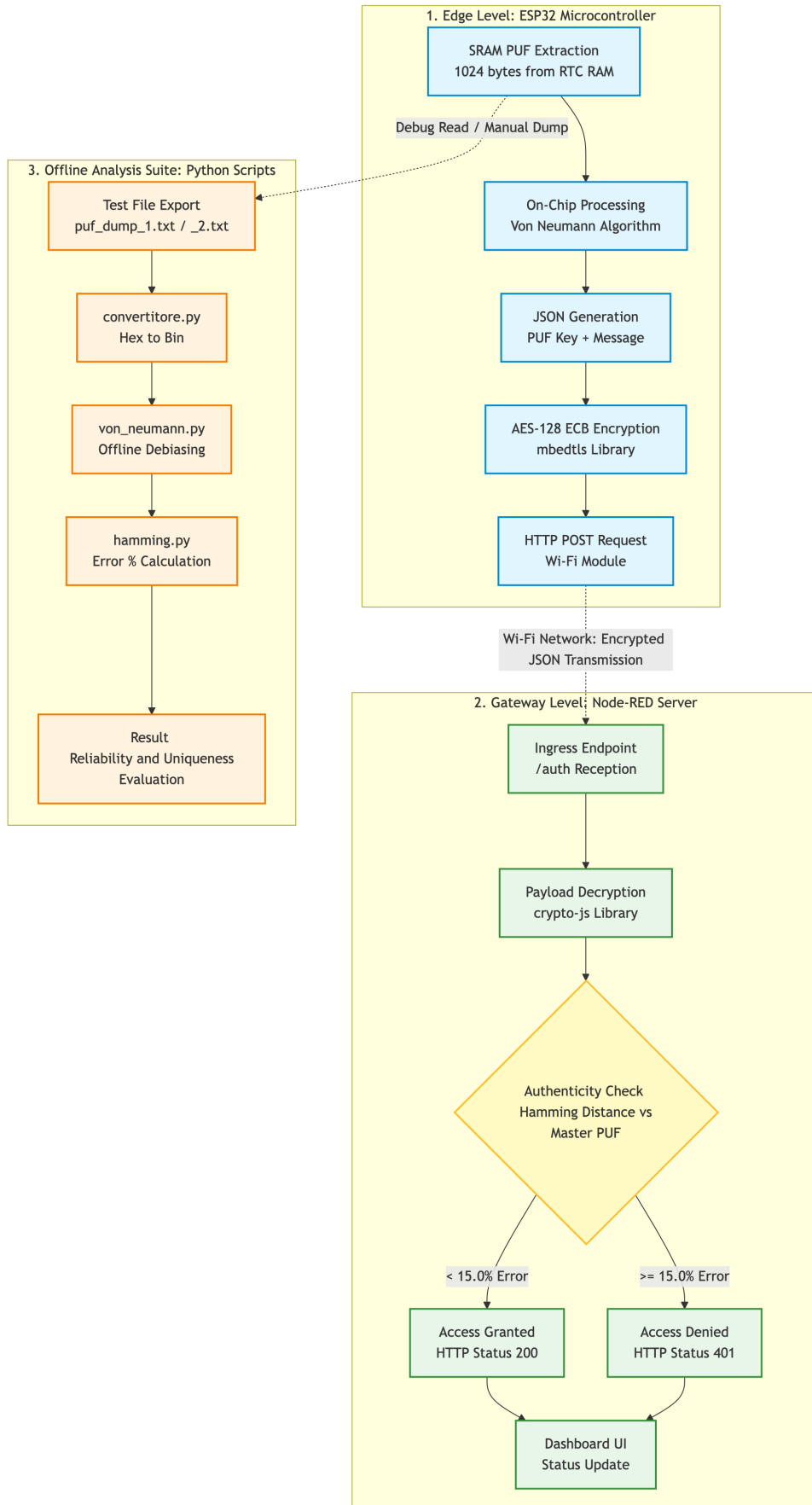


Figure 1: System Architecture Diagram

- **Access Logic and UI:** If the error percentage (distance) is lower than the 15.0% tolerance threshold, the device is recognized as authentic, the server returns an HTTP 200 status, and the dashboard ("IoT Security") updates the interface to show "ACCESS GRANTED" along with the decrypted message. Otherwise, access is denied (HTTP 401), identifying a potential clone.

2.3 Offline Analysis Suite (Python Scripts)

To verify the cryptographic quality of the generated keys, the architecture is supported by a set of Python scripts (located in the `Python_IoT_security` directory), which are essential for hardware data analysis.

- **convertitore.py:** Takes text files containing raw hexadecimal memory dumps (`puf_dump_1.txt`, `puf_dump_2.txt`) as input and converts them into binary streams without information loss.
- **von_neumann.py:** Simulates the processing performed by the ESP32 offline. It reads the raw binary stream, applies the Von Neumann logic (mapping 01 to 0 and 10 to 1), generates a purified output file (e.g., `puf_dump_vn_2.txt`), and calculates the extraction yield percentage.
- **hamming.py:** Calculates the Hamming Distance between two purified binary files. This script evaluates reliability (Intra-Distance) if the error is less than 40% (same chip) and uniqueness (Inter-Distance) if the error is between 40% and 60% (different chips), providing essential diagnostic analysis for the academic phase of the project.

3 Data Flow and Security

The project's security pipeline is designed to protect the edge node's identity at every stage of the authentication process, from hardware fingerprint generation to server-side validation. The data flow is divided into four sequential phases:

3.1 Hardware Purification

The process begins at the hardware level the exact moment the device is powered on (Cold Boot).

- **Raw Extraction:** The ESP32 extracts a 1024-byte block of raw data from its RTC memory (declared with the `RTC_NOINIT_ATTR` attribute), an area protected from automatic overwriting during system initialization.
- **Von Neumann Debiasing:** Because SRAM memories tend to have a physical bias (a natural predisposition toward zero or one), the extracted raw hexadecimal data is converted to binary and processed internally by the microcontroller using the Von Neumann algorithm.
- **Purification Logic:** The algorithm iterates through bits in pairs: 01 pairs are translated into a single 0, 10 pairs generate a 1, while 00 and 11 pairs are discarded entirely. This ensures the final key is statistically balanced and extracts only the silicon's true entropy.

3.2 Payload Encapsulation

Once the PUF key is generated (which is purely volatile and never permanently stored in Flash memory), the device prepares the network packet.

- **Message Composition:** An original JSON file is generated containing the binary string of the calculated PUF and a payload message (e.g., `"message": "Temperature Alarm!"`).
- **Symmetric Encryption:** The plaintext JSON undergoes standard AES-128 encryption. The algorithm operates in ECB (Electronic Codebook) mode, applying PKCS#7 padding to adjust the text length to the cryptographic block size.
- **Pre-Shared Key (PSK):** The encryption process utilizes the `mbedtls` library and a fixed 16-byte secret key (in the proof of concept: `SuperSegreta1234`).
- **Final Obfuscation:** The encrypted output (byte array) is converted into a hexadecimal string and wrapped in a minimal new JSON with the key `"dati_sicuri"`, thereby completely hiding the original payload structure from any potential network "sniffers."

3.3 Transmission and Decryption

- **Network Transmission:** The secure packet is forwarded via Wi-Fi to the server through an HTTP POST request directed to the Node-RED `/auth` endpoint.
- **Server-Side Reversing:** Upon arrival, the Node-RED function node retrieves the content of `dati_sicuri`. Using the JavaScript library `crypto-js`, the server decodes the hexadecimal string and applies the inverse cryptographic process (AES-128 ECB, PKCS#7 padding) using the same shared secret key.
- **Data Recovery:** The operation reconstructs the original string, which is then parsed to return the JSON object containing the cleartext PUF and the associated test message.

3.4 Fault-Tolerant Validation (Hamming Distance)

PUF responses extracted from the same chip at different times exhibit natural electronic noise (some bits change between reboots). Therefore, the server cannot perform a simple equality check.

- **Error Calculation:** The server isolates the newly arrived PUF and compares it with a trusted "Master" PUF fingerprint, stored securely at the backend level. The comparison is performed by calculating the Hamming Distance: the number of discordant bits between the two strings is counted. The calculation is performed up to the length of the shorter string to avoid buffer mismatches (`Math.min`).
- **Decision Logic:** The detected distance is converted into a percentage of discrepancy.
 - If the error is below the pre-set 15.0% tolerance threshold, the server accepts the differences as intrinsic noise, validates the device identity, grants access (HTTP 200 OK), and displays the data on the Dashboard.

- If the limit is exceeded, the server recognizes that the fingerprint belongs to a different physical silicon (or indicates an attack/cloning attempt) and blocks access, returning an HTTP 401 (Access Denied).

4 Setup and Deployment Guide

To facilitate the replication of this architecture, this section provides a roadmap for setting up the hardware, software, and validation components of the PUF-based IoT system.

4.1 Hardware and Software Prerequisites

Hardware: ESP32 microcontroller, USB cable for programming, and a host computer for the Node-RED environment and offline analysis.

ESP32 Libraries: The `PUF_starter_1.ino` file requires standard `Arduino.h`, `WiFi.h`, and `HTTPClient.h` libraries, alongside the native `mbedtls` cryptographic suite.

Server Environment (Node-RED): It is necessary to install the `crypto-js` library in the function node to allow for the decryption of AES-128 payloads. The dashboard relies on the `@flowfuse/node-red-dashboard` package (version 1.30.1 or higher).

4.2 Deployment Procedure

1. **Edge Node Configuration:** Open `PUF_starter_1.ino`, configure the `ssid` and `password` for your Wi-Fi network, and set `serverName` to the local IP address of the computer hosting Node-RED (port 1880). Flash the code onto the ESP32 using the Arduino IDE.
2. **Gateway Setup:** Import the `flows-2.json` file into your Node-RED instance. Verify that the `http in` node is correctly configured on the `/auth` path.
3. **Master PUF Configuration:** In the Node-RED `function 1` node, copy and paste the binary string of the PUF extracted during the device's first initialization into the `masterPuf` constant.
4. **Offline Analysis Execution:** To use the analysis tools, place the dump files (`puf_dump_1.txt`, etc.) into the `Python_IoT_security` folder and run the scripts in order (`convertitore.py` → `von_neumann.py` → `hamming.py`) to generate the validation reports.

5 Known Issues and Future Developments

Aligned with our iterative development methodology, this section outlines current system limitations and prioritizes the backlog for subsequent development cycles.

5.1 Known Issues (Current State)

- **Empirical Validation:** The 15.0% authentication threshold was calculated based on limited preliminary tests; it requires broader statistical validation across a dataset of heterogeneous devices.
- **Encryption Mode:** The use of AES-ECB mode was chosen for its implementation lightweightness on microcontrollers, but it is less secure than modern modes like AES-GCM or CBC, which offer protection against ciphertext manipulation.
- **Cold Boot Dependency:** The system relies on the ESP32's ability to preserve SRAM state upon reboot; in extreme temperature conditions, electronic noise may vary, affecting the Hamming Distance percentage.

5.2 Next Steps (Backlog for Upcoming Sprints)

- **Cryptographic Optimization:** Migration towards authenticated encryption modes (e.g., AES-GCM) to ensure message integrity in addition to confidentiality.
- **Test Automation:** Implementation of an automated test suite for real-time, large-scale Hamming Distance calculation, reducing dependency on offline Python scripts.
- **Dynamic Thresholding:** Development of a server-side algorithm capable of dynamically adjusting the Hamming tolerance threshold based on the device's reading history, improving resilience to thermal noise.